Tìm hiểu về Retrieval Augmented Generation (RAG)

Editors' Choice

Tổng quan

Chúng ta có thể hiểu nôm na RAG trong thời đại LLMs như sau:

- Retrieval-Augmented Generation (RAG) là một kỹ thuật giúp nâng cao khả năng của mô hình sinh (language model generation) kết hợp với tri thức bên ngoài (external knowledge)
- Phương pháp này thực hiện bằng cách truy xuất thông tin liên quan từ kho tài liệu (tri thức) và sử dụng chúng cho quá trình sinh câu trả lời dựa trên LLMs.

Sự nổi lên của RAG

- Trong sự ra đời của các mô hình ngôn ngữ lớn (LLMs) như ChatGPT, LLama-2, Qwen, Mistral,... rất hay gặp vấn đề ảo giác, trong từ điển chuyên ngành gọi là *hallucination*, khi đó các mô hình sinh ra những câu văn nghe rất trôi chảy và hợp lý, nhưng trong thực tế nó lại không chính xác. Chính vì đó là nguồn cảm hứng cho việc kỹ thuật RAG ra đời và như là một phương pháp để giải quyết vấn đã nêu.
- Trong nhiều trường hợp, khi các doanh nghiệp, công ty có những nguồn tài liệu độc quyền, chẳng hạn như tài liệu sổ tay kỹ thuật, tài liệu hướng dẫn sử dụng của các sản phẩm của họ, ... Với các dạng tài liệu lớn và riêng biệt như vậy, việc yêu cầu các mô hình ngôn ngữ lớn trích xuất thông tin cụ thể từ nội dung đồ sộ này chẳng khác nào là mò kim đáy bể.
- Gần đây, OpenAI cũng giới thiệu mô hình mới là GPT-4 có khả năng xử lý các tài liệu lớn, có khả năng giải quyết nhu cầu này. Tuy nhiên, mô hình này không hoàn toàn hiệu quả do hiện tượng mất đi những thông tin context ở giữa (Lost in its the middle).
- Để giải quyết các vấn đề này, một phương pháp thay thế được gọi là Retrieval-Augmented Generation (RAG) được phát triển. Phương pháp này liên quan tới việc tạo chỉ mục cho mỗi đoạn văn trong tài liệu (document). Khi một truy vấn (query) được thực hiện sẽ truy xuất các đoạn văn liên quan nhất và sau đó đưa vào mô hình ngôn ngữ lớn như ChatGPT, GPT-4, ... Chiến lược chỉ cung cấp các đoạn văn có chọn lọc, thay vì toàn bộ tài liệu ngăn chặn tình trạng quá tải thông tin cho LLM và nâng cao chất lượng đáng kể.

Mô hình Neural Retrieval

Trước khi nhảy vào hiểu về RAG, chúng ta dành thời gian hiểu chút một cách tổng quan về mô hình neural retrieval.

- Mô hình Neural Retrieval là một loại mô hình truy xuất thông tin sử dụng mạng nơ-ron (neural network) để tìm kiếm các đoạn văn (chunks) trong tài liệu (documents) liên quan dựa trên câu truy vấn (query). Chúng mã hóa (encode) câu query và các chunks trong các documents thành các biểu diễn vector, gọi là dense vector và tính toán mức độ tương đồng giữa chúng. Điều này cho phép chúng nắm bắt được sự liên quan về cả ngữ nghĩa thay chỉ vì sự kết hợp từ vựng với nhau.
- Neural Retrieval là sự phát triển đáng kể từ các hệ thống retrieval dựa trên từ khóa (keyword-based) sang hệ thống truy xuất thông tin hiểu được ý nghĩa và mối quan hệ ngữ nghĩa cơ bản trong dữ liệu.
- Cách Neural Retriever hoạt động:
    1. **Vector Encoding:**
        - Cả queries và chunks được biến đổi thành vector trong không gian nhiều chiều. Quá trình này được thực hiện bởi bộ mã hóa (encoders) dựa trên neural-network đã được huấn luyện (training) để nắm bắt bản chất ngữ nghĩa của văn bản text.
        - Trong quá trình training, những mô hình này thường được tiếp xúc với lượng văn bản lớn, cho phép chúng tìm hiểu các mẫu (patterns) và mối quan hệ giữa các từ và cụm từ.
    2. **Semantic Matching:**
        - Độ tương đồng giữa các vector của query và chunks được tính toán bằng cách sử dụng các thước đo chẳng hạn như độ tương đồng cosine. Điều này cho phép hệ thống xác định document nào liên quan nhất tới câu query dựa trên ý nghĩa của nội dung thay vì sự trùng lặp về từ khóa.
        - Quá trình này có thể nắm bắt các mối quan hệ sắc thái như từ đồng nghĩa, hoặc khái niệm liên quan, mà các phương pháp truyền thống bỏ xót.
- **Ưu điểm của Neural Retriever:**
    - Neural Retriever có thể hiểu ngữ cảnh sử dụng các thuật ngữ, cho phép truy xuất chính xác hơn khi queries và chunks có nghĩa không rõ ràng hoặc đa nghĩa.

    - Chúng có khả năng giải quyết với các câu query dài và phức tạp vì chúng có thể nắm bắt được ý định tổng thể thay vì chỉ là các thuật ngữ riêng lẻ.
    - Nhiều mô hình Neural Retiever được huấn luyện trên các bộ dữ liệu đa ngôn ngữ (multilingual), cho phép chúng xử lý các câu query trong nhiều ngôn ngữ hiệu quả.
- **Thách thức:**
    - Các mô hình nơ-ron, đặc biệt là các mô hình được sử dụng để encode tài liệu lớn đòi hỏi sức mạnh tính toán đáng kể cho quá trình huấn luyện (training) và suy luận (inference).
    - Hiệu suất của mô hình Neural Retriever phụ thuộc rất nhiều vào dữ liệu mà chúng được huấn luyện và chúng có khả năng kế thừa những sai lệch có trong dữ liệu huấn luyện.
    - Việc lưu trữ các biểu diễn cho các chunks trong các documents luôn được cập nhật là một thách thức, dặc biệt các nội dung thay đổi linh hoạt.

Retrieval-Augmented Generation (RAG) Pipeline

- Với RAG, LLMs có thể tận dụng tri thức và thông tin mà không nhất thiết phải được lưu trữ trong trọng số mô hình của nó (tức là không chỉ dựa vào những gì nó đã học) bằng cách cung cấp các nguồn tri thức bên ngoài, chẳng hạn như database, sách, báo, website, ...
- Điều này thúc đẩy công cụ truy xuất thông tin (retriever) để tìm kiếm các tri thức liên quan để điều chỉnh LLMs, bằng cách này RAG có thể nâng cao nền tảng tri thức của LLMs với nguồn tri thức bên ngoài.
- Công cụ truy xuất (retriever) ở đây có thể là bất kỳ công cụ nào sau đây tùy thuộc vào nhu cầu về ngữ nghĩa hay không:
    - **Vector database:** Thông thường, các câu query được embed sử dụng mô hình như BERT (Transformers-based) để tạo ra dense vector embedding. Ngoài ra, các phương pháp truyền thống như TF-IDF có thể được sử dụng như là sparse embeddings. Việc tìm kiếm sau đó được tiến hành dựa trên độ tương đồng về ngữ nghĩa (semantic similarity) hoặc tần suất các thuật ngữ (term frequency).
    - **Graph database:** Xây dựng cơ sở tri thức (knowledge base) từ mối quan hệ giữa các thực thể (entity) được trích xuất trong văn bản. Cách tiếp cận này đảm bảo tri

thức tính xác, nhưng đòi hỏi truy vấn chính xác, điều này cũng sẽ hạn chế trong một số ứng dụng.

    - **Regular SQL database:** Cung cấp lưu trữ và truy xuất dữ liệu có cấu trúc nhưng có thể thiếu linh hoạt về mặt ngữ nghĩa.

- Hình ảnh dưới nói về sự khác biệt giữa việc sử dụng Graph database và Vector database trong RAG. Graph database được cho là ưu tiên hơn cho RAG khi so với Vector database bởi nó chứa những tri thức chính xác cao. Trong khi Vector database chia và indexing dữ liệu sử dụng vector dựa trên LLM để encode, cho phép có khả năng truy xuất thông tin dựa trên ngữ nghĩa nhưng chúng có thể đi kèm theo những thông tin không liên quan dẫn tới sai lệch có thể sảy ra. Mặc khác, Graph database xây dựng một knowledge base từ các mối quan hệ giữa các thực thể được trích xuất trong văn bản, giúp cho việc truy xuất thông tin trở nên ngắn gọn, tuy nhiên đòi hỏi việc các truy vấn phải chính xác với cách dữ liệu được lữu trữ và liên kết. Tức là nếu chúng ta không biết chính xác cách thông tin được cấu trúc và liên kết sẽ gặp khó khăn trong việc truy xuất thông tin mà chúng ta cần.

- Sau khi retrieval, chúng ta có thể xem xét lọc tri thức liên quan bằng cách thêm các lớp ranking cho phép lọc ra những tri thức không phù hợp với quy tắc mà chúng ta muốn, hoặc không được cá nhân hóa cho người dùng, bối cảnh hiện tại hoặc giới hạn về độ dài.
- Tóm tắt ngắn gọn quá trình của RAG như sau:
    1. **Create Vector database:** Đầu tiên, convert toàn bộ dữ liệu tri thức thành các vector và lưu trữ chúng vào một vector database.
    2. **User input:** User cung cấp 1 câu truy vấn (query) bằng ngôn ngữ tự nhiên nhằm tìm kiếm câu trả lời hoặc để hoàn thành câu truy vấn đó.
    3. **Information retrieval:** Cơ chế retrieval quét toàn vộ vector trong database để xác định các phân đoạn tri thức (chính là paragraphs) nào có ngữ nghĩa tương đồng với câu truy vấn của người dùng. Các paragraphs này sau đó được vào LLM để làm tăng context cho quá trình sinh ra câu trả lời.
    4. **Combining data:** Các paragraphs được lấy sau quá trình retrieval từ database được kết hợp với câu query ban đầu của user tạo thành 1 câu prompt.
    5. **Generate text:** Câu prompt được bổ sung thêm context sau đó được đưa qua LLM để sinh ra câu phản hồi cuối cùng theo context bổ sung.
- Hình ảnh sau minh họa quá trình hoạt động của RAG:

Ưu điểm của RAG

Tại sao chúng ta nên sử dụng RAG trong việc phát triển các ứng dụng. Hay nói các khác RAG là giải pháp mà các công ty, doanh nghiệp hướng đến trong phát triển sản phẩm trong thời đại LLMs:

- Với RAG, LLM có thể tận dụng dữ liệu bên ngoài để cung cấp tri thức cho nó.
- RAG không yêu cầu training lại mô hình, tiết kiệm thời gian và tài nguyên tính toán.
- Nó hiệu quả ngay cả với một lượng dữ liệu gán nhãn hạn chế.
- Tuy nhiên, RAG cũng có nhược điểm đó là hiệu suất của RAG phụ thuộc vào chất lượng độ chính xác của model retrieval; tính toàn diện và chính xác của kho tri thức có sẵn.

- RAG phù hợp nhất cho các tình huống có nhiều dữ liệu chưa được gán nhãn (unlabeled-data) nhưng nguồn dữ liệu gán nhãn khan hiếm và lý tưởng cho các ứng dụng như trợ lý ảo cần truy cập theo thời gian thực vào thông tin cụ thể như hướng dẫn sử dụng phần mềm, tin tức, ...
    - Trường hợp mà có nhiều dữ liệu unlabeled-data nhưng lại khan hiếm dữ liệu labeled-data: RAG rất hữu ích trong những trường hợp này, tức là có sẵn nhiều dữ liệu nhưng hầu hết dữ liệu đó không được phân loại hoặc gãn nhãn theo cách hữu ích cho mô hình có thể học. Ví dụ như internet có số lượng tin tức, văn bản text lớn nhưng hầu hết các văn bản text đó không được tổ chức theo các trả lời trực tiếp các câu hỏi cụ thể.
    - RAG lý tưởng cho các ứng dụng như trợ lý ảo, chatbot: Các trợ lý ảo, chatbot, như Siri hay Alexa cần lấy thông tin từ nhiều nguồn khác nhau để trả lời các câu hỏi khác nhau trong thời gian thực. Chúng cần hiểu câu hỏi, lấy thông tin liên quan và sau đó đưa ra câu trả lời mạch lạc và chính xác.
    - Cần truy cập theo thời gian thực (real-time) vào thông tin cụ thể như hướng dẫn sử dụng: Đây là 1 ví dụ về trường hợp mà RAG đặc biệt hữu ích. Tưởng tượng là bạn hỏi trợ lý ảo một câu hỏi cụ thể về một sản phẩm, chẳng hạn như "Làm thế nào để reset điều khiển ABC". RAG trước tiên sẽ truy xuất thông tin liên quan từ hướng dẫn sử dụng sản phẩm hoặc các tài liệu khác, sau đó sử dụng thông tin đó để tạo ra câu trả lời rõ ràng, ngắn gọn.
    - Tóm lại, các mô hình RAG rất phù hợp cho các ứng dụng có sẵn nhiều thông tin nhưng không được tổ chức hay gán nhãn phù hợp cho mô hình có thể học.

Lựa chọn Vector database

- Có rất nhiều công cụ Vector database để tích hợp vào hệ thống RAG. Ảnh dưới mô tả các loại và so sánh một số ưu điểm các loại Vector database phổ biến.

- Mình có tìm được 1 nguồn so sánh các Vector DB khá chi tiết, bạn có thể xem ở đây.

Xây dựng một hệ thống RAG

- Một góc nhìn tổng quan thì RAG có 3 bước: Ingestion (Biến đổi dữ liệu), Retrieval (Truy xuất thông tin), và Synthesis/ Response Generation (Tổng hợp và sinh câu trả lời)

- Trong các phần tiếp theo dưới đây, chúng ta sẽ đi vào tìm hiểu về 3 steps này.

Ingestion

Ingestion là quá trình có thể hiểu chung là biến đổi dữ liệu, về cơ bản nó gồm các quá trình:

- Thu thập dữ liệu
- Tiền xử lý dữ liệu
- Lập chỉ mục và lưu trữ vào database (Indexing/ Embedding/ Storage DB)

Chunking

- Chunking (phân đoạn) là quá trình chia prompts hoặc documents thành nhiều chunks, hoặc gọi là segments nhỏ hơn nhưng vẫn có nghĩa. Các chunks này có thể được cắt theo một kích thước cố định, chẳng hạn như số lượng ký tự, câu hoặc đoạn văn cụ thể.
- Trong RAG, mỗi chunk được mã hóa (encode) thành 1 vector embedding để retrieval. Phân tách các chunks đủ nhỏ, chính xác sẽ giúp kết quả truy xuất thông tin giữa câu truy vấn của user và nội dung chunk chính xác hơn.
- Việc phân tách chunks quá lớn có thể bao gồm nhiều thông tin không liên quan, gây nhiễu và có khả năng làm giảm độ chính xác của retrieval. Bằng các việc kiểm soát kích thước của các chunks, RAG có thể cân bằng giữa lượng thông tin đầy đủ và tính chính xác.
- **Thế làm thế nào để chọn kích thước chunks phù hợp cho trượng hợp cụ thể?** Việc lựa chọn kích thước chunking trong RAG là rất quan trọng. Nó cần đủ nhỏ để đảm bảo tính liên quan và giảm nhiễu nhưng cũng cần đủ lớn để duy trì được đủ thông tin, giữ được context. Có 1 cách mà người làm AI vẫn luôn làm "thử và đánh giá". Bạn có thể xem xét một số phương pháp (tham khảo từ Pinecone) sau đây:
    - **Fixed-size chunking:** Đơn giản là lựa chọn số lượng tokens trong 1 chunk và xem xét có nên overlap giữa chúng không? Overlap giữa các chunks để hạn chế việc mất context giữa các chunks. Cách này đơn giản về mặt tính toán và dễ dàng thực hiện.
    - text = "..." # your text
    - from langchain.text_splitter import CharacterTextSplitter
    - text_splitter = CharacterTextSplitter(
    - separator = "\n\n",

- chunk_size = 256,
- chunk_overlap = 20
-)
- docs = text_splitter.create_documents([text])
- **Context-aware chunking:** Context-aware chunking tận dụng cấu trúc bên trong của văn bản để phân tách các chunks tạo ra các chunks có ý nghĩa hơn và phù hợp với ngữ cảnh hơn. Cách này cũng được mình áp dụng trong việc phát triển các sản phẩm liên quan đến hệ thống này. Chúng ta sẽ xem một số cách tiếp cận sau.

    1. **Sentence Splitting:** Phương pháp này phù hợp với các models tối ưu cho embedding nội dung văn bản theo mức độ câu (sentence-level). Các kỹ thuật khác nhau có thể sử dụng để tách câu:

        - **Naive Splitting:** Phương pháp cơ bản nhất trong đó câu được phân tách bằng dấu chấm hoặc dấu xuống dòng. Phương pháp này đơn giản, nhanh chóng nhưng có thể sẽ bị bỏ qua các câu có cấu trúc phức tạp làm mất context của chúng.
        - text = "..." # Your text
        - docs = text.split(".")
        - **NLTK (Natural Language Toolkit):** NLTK là một thư viện phổ biến, toàn diện để xử lý ngôn ngữ tự nhiên. NLTK có bộ sentence tokenizer giúp phân tách văn bản text thành các câu một cách hiệu quả.
        - text = "..." # Your text
        - from langchain.text_splitter import NLTKTextSplitter
        - text_splitter = NLTKTextSplitter()
        - docs = text_splitter.split_text(text)
        - **spaCy:** spaCy là một thư viện nâng cao cho các tác vụ trong NLP, cung cấp khả năng phân đoạn các câu hiệu quả.
        - text = "..." # Your text
        - from langchain.text_splitter import SpacyTextSplitter

- text_splitter = SpacyTextSplitter()
- docs = text_splitter.split_text(text)
- **Vncorenlp:** Vncorenlp là một thư viện phổ biến sử dụng cho Tiếng Việt. Cũng tương tự như NLTK & spaCy, Vncorenlp có bộ tokenizer giúp phân tách văn bản text thành các cụm từ, các câu một cách hiệu quả và được sử dụng rộng rãi trong cộng đồng NLP Tiếng Việt. Bạn có thể tham khảo cách sử dụng github VnCoreNLP.

2. **Recursive Chunking:** Recursive chunking là một phương pháp giúp phân tách văn bản text theo phân tầng bằng cách sử dụng các dấu phân tách khác nhau. Nó thích ứng để tạo ra các chunks có kích thước, hoặc cấu trúc tương tự nhờ sử dụng phương pháp đệ quy với các tiêu chí khác nhau. Một ví dụ ở trong Langchain:

```
text = "..."  # Your text
from langchain.text_splitter import RecursiveCharacterTextSplitter
text_splitter = RecursiveCharacterTextSplitter(
    chunk_size = 256,
    chunk_overlap = 20
)
docs = text_splitter.create_documents([text])
```

10. **Specialized Chunking:** Đối với dữ liệu dạng như Markdown, hoặc LaTeX có thể sử dụng chunking riêng biệt cho nó để giữ được cấu trúc ban đầu.
    - **Markdown Chunking:** Nhận biết được cú pháp Markdown và phân tách nội dung văn dựa trên cấu trúc. Ví dụ cách sử dụng:

```
from langchain.text_splitter import MarkdownTextSplitter
markdown_text = "..."
markdown_splitter = MarkdownTextSplitter(chunk_size=100, chunk_overlap=0)
docs = markdown_splitter.create_documents([markdown_text])
```

    - **LaTeX Chunking:** Cho phép phân tích các command trong LaTeX để phân tách nội dung trong khi vẫn giữ được cấu trúc logic của nó.

Nói chung, theo một mindset của mình thì để chunking một văn bản, tài liệu hiệu quả thì "nếu đoạn văn bản được chunking có ý nghĩa mà không cần đến ngữ cảnh xung quanh đối với con người thì nó cũng sẽ có ý nghĩa đối với mô hình ngôn ngữ, và mô hình ngôn ngữ có thể hiểu." Đó theo mình nghĩ nếu đáp ứng được yêu cầu vậy thì có thể nói là chúng ta đã có 1 giải pháp chunking 1 tài liệu hiệu quả.

Embeddings

- Sau khi chúng ta chunking documents, bước tiếp theo là embed chúng. Embed documents trong RAG tức là biến đổi cả câu query của user và các documents trong kho tri thức thành một định dạng, một không gian vector để có thể so sánh về mức độ liên quan giữa chúng. Quá trình này rất quan trọng đối với khả năng của RAG trong việc truy xuất thông tin phù hợp nhất từ kho tri thức với câu query của user.
- Để lựa chọn mô hình embedding phù hợp cho bài toán của chúng ta cũng là cần thiết. Thông thường, mình sẽ dựa vào Benchmark của embedding model trên các bộ dữ liệu lớn cho các tác vụ cụ thể. Các bạn có thể xem và tham khảo như ở HuggingFace's Massive Text Embedding Benchmark (MTEB) leaderboard. Có 1 câu hỏi đặt ra là nên sử dụng loại embedding nào? Dense embedding hay Sparse embedding. Chúng ta cùng phân tích xem lợi ích của từng loại.
    - **Sparse embedding:** Sparse embedding chẳng hạn như TF-IDF rất phù hợp để so khớp (matching) từ vựng giữa câu query và documents. Ứng dụng tốt nhất cho trường hợp là mức độ liên quan giữa các từ khóa (keyword) là rất quan trọng. Nó ít đòi hỏi chi phí tính toán chuyên sâu hơn nhưng có thể không nắm bắt được ngữ nghĩa sâu sắc.
    - **Semantic embedding:** Semantic embedding/ Dense embedding chẳng hạn như các mô hình transformers-based: BERT, SentenceBERT, ... thường được sử dụng nhiều do khả năng nắm bắt ngữ nghĩa của câu. Loại này thường được sử dụng nhiều trong hệ thống RAG. Cơ bản hiện nay có rất nhiều loại kiến trúc mô hình embedding, chính vì thế chúng ta có thể cần nhìn nhận xem với ứng dụng của mình nên sử dụng mô hình kiến trúc nào, dựa vào MTEB như mình có trích dẫn trên cũng là một cách mình nhìn nhận và trực giác được khả năng của từng loại mô hình.
- Ở trong bài viết này mình sẽ nói thêm về Sentence Transformers, bởi vì chúng lý tưởng cho các trường hợp trong đó yêu cầu mô hình nắm bắt được ngữ nghĩa và câu có ý nghĩa. Nó tạo ra sự cân bằng giữa sự nắm bắt tốt ngữ cảnh như BERT và yêu cầu biểu diễn câu có ý nghĩa, chính vì thế được ưa thích ứng dụng trong RAG.

- **Bây giờ chúng ta đặt câu hỏi: Sentence Embedding là cái gì? và Tại sao nó lại được lựa chọn**
    - **Background: Sự khác biệt giữa Sentence Transformers so với các mô hình dạng Token-level chẳng hạn như BERT**
        - Sentence Transformers cũng có thể xem như là một biến thể của BERT, được thiết kế kiến trúc đặc biệt để tạo ra embedding cho toàn bộ câu (sentence-level), tức là sentence embedding. Điểm khác biệt chính nằm trong phương pháp đào tạo của chúng:
            1. **Objective:** BERT được huấn luyện để dự đoán các từ (words) được ẩn đi (masking) trong 1 câu và dự đoán câu tiếp theo. Đó chính là 2 tasks mà nếu bạn đã đọc paper về BERT có thể đã biết: Masked prediction và Next sentence prediction. Mặc khác, Sentence Transformers được đặc biệt huấn luyện để hiểu ý nghĩa của toàn bộ câu. Chúng tạo ra các embeddings trong đó các câu có ý nghĩa tương tự nhau sẽ nằm gần nhau trong 1 không gian vector. Sentence Transformers sử dụng Siamese or triplet networks. Trong một cấu trúc siamese, mô hình được huấn luyện để tối ưu hóa khoảng cách giữa các cặp câu tương tự nhau. Trong khi đó, mạng triplet bao gồm một cặp "anchor" và "positive" (các câu tương tự nhau) và một câu "negative" (không tương tự). Mục tiêu là tối ưu hóa khoảng cách sao cho anchor và positive gần nhau hơn so với anchor và negative. Bạn có thể tìm hiểu về Contrastive learning để hiểu hơn về nó.
            2. **Level embedding:** Điểm khác biệt chính cũng nằm ở level của embedding. BERT cung cấp embedding cho từng tokens (word hoặc subword) trong 1 câu, trong khi Sentence Transformers đưa ra 1 vector embedding duy nhất cho cả câu đó.
            3. **Training Data và Tasks:** BERT chủ yếu được huấn luyện trên tập dữ liệu lớn với tasks tập trung vào việc hiểu các từ trong ngữ cảnh, trong khi Sentence Transformers thường được huấn luyện trên tập dữ liệu bao gồm các cặp câu. Quá trình huấn luyện này tập trung vào mức độ tương đồng và sự liên quan, dạy cho model cách hiểu và so sánh ý nghĩa của cả câu.
            4. **Fine-Tuning for Specific Tasks:** Sentence Transformers thường được fine-tuning cho các tasks cụ thể chẳng hạn như Semantic

Similarity, Paraphrase identification hay là Retrieval Information. Việc fine-tuning này tập trung hơn vào việc hiểu nắm bắt nghĩa cả câu (sentence-level). Trong khi BERT thường được fine-tuning cho nhiều tasks hơn như Question Answering, Classification, NER, ... tập trung vào mức độ từ/ cụm từ (word-level/ phrase-level).

5. **Applicability:** BERT và các mô hình tương tự linh hoạt hơn cho các tasks mà thiên về nắm bắt và hiểu ở mức độ từ, chẳng hạn như Named entities recognition, Question answering), trong khi Sentence Transformers phù hợp cho các tasks mà đòi hỏi việc phải hiểu ý nghĩa ở mức độ câu, chẳng hạn như Semantic search, Sentence similarity.

6. **Efficiency in Generating Sentence Embedding or Similarity Tasks:** Trong BERT, việc tạo ra 1 vector embedding cho cả câu thường sử dụng hidden states của token đầu tiên trong câu (trong BERT là [CLS] làm biểu diễn cho toàn bộ câu, hoặc có nhiều thử nghiệm cho việc tạo ra vector embedding cho cả câu dựa trên embedding của các tokens trong câu đó và qua các phép như pooling, evarage, concat, ... Tuy nhiên, cách làm này không phải lúc nào cũng tối ưu, hiệu quả cho các tasks kiểu dạng sentence-level. Sentence Transformers đặc biệt được thiết kế để tối ưu để tạo ra các vector embedding cho cả câu chứa ý nghĩa, ngữ nghĩa hữu ích hơn, do đó hiệu quả hơn cho các tasks liên quan tới tính toán mức độ tương đồng giữa các câu. Vì chúng chỉ tạo ra 1 vector embedding duy nhất cho mỗi câu, nên việc tính toán similarity score sẽ ít tốn kém hơn so với các model dạng token-level.

Tóm lại, BERT là mô hình biểu diễn ngôn ngữ cho nhiều mục đích chung, tập trung vào ngữ cảnh theo word-level, còn Sentence Transformers được điều chỉnh đặc biệt để hiểu và so sánh ý nghĩa của cả câu, làm cho chúng hiệu quả hơn đối với các nhiệm vụ đòi hỏi nắm bắt được ngữ nghĩa của toàn bộ câu.

- **Related: Quá trình huấn luyện Sentence Transformers và Token-level Embedding Models**
    - Chúng ta sẽ tìm hiểu xem cách Sentence Transformers được huấn luyện khác biệt gì so với các mô hình token-level embedding (BERT).
    - Chúng ta đã nói về câu chuyện Sentence Transformers được huấn luyện để tạo ra các embedding hiệu quả cho embed câu ở trên, đây là một cách

tiếp cận khác biệt so với các mô hình tạo ra để embed cho token-level như BERT. Mình sẽ mô tả tổng quan quá trình huấn luyện cú nó để xem khác biệt gì so với Token-level models:

1. **Model Architecture:** Sentence Transformers thường bắt đầu bằng một mô hình base chẳng hạn như sử dụng BERT hoặc các kiến trúc mô hình transformers-based khác làm mô hình base cho nó. Tuy nhiên, nó tập trung tâm vào câu chuyện là làm sao để đưa ra một vector embedding duy nhất cho toàn bộ câu thay vì từng token riêng.

2. **Training data:** Chúng được huấn luyện trên nhiều tập dữ liệu lớn khác nhau, thường bao gồm các cặp câu hoặc nhóm câu trong đó có nhãn để biết được mối quan hệ giữa các câu (similarity, paraphrasing).

3. **Training objectives:** BERT được huấn luyện với dạng Masked language model (dự đoán từ bị thiếu/ che đi) và Next sentence prediction, tập trung hiểu ngữ cảnh ở mức độ token/ word (token-level). Mặc khác, Sentence Transformers được huấn luyện để hiểu ngữ cảnh và mối quan hệ ở mức độ câu (sentence-level). Training objective của nó là giảm thiểu khoảng cách giữa các vector embeddings tương tự về mặt ngữ nghĩa trong khi tối đa hóa khoảng cánh giữa các vector embbedings của những câu khác nhau, đối lập nhau về ngữ nghĩa. Điều này đạt được nhờ thông qua các hàm contrastive loss như triplet loss, cosine similarity loss, ...

4. **Output representation:** Trong BERT, biểu diễn cho câu (sentence-level representation) thường sử dụng embedding của token [CLS] đứng đầu tiên ở mỗi câu hoặc bằng cách sử dụng các toán tử pooling, average, concatenating, ... của các token embeddings. Sentence Transformers được thiết kế để trực tiếp đưa ra một vector embedding biểu diễn cho sentence-level.

5. **Fine-tuning for Downstream tasks:** Sentence Transformers có thể được fine-tune trên các tác vụ cụ thể chẳng hạn như Semantic text similarity, trong đó mô hình học cách để tạo ra các embedding nắm bắt được sắc thái ngữ nghĩa của cả câu.

        - Tóm lại, Sentence Transformers được nghiên cứu và hình thành để tối ưu cho biểu diễn ở mức độ câu, nắm bắt ngữ nghĩa tốt 1 cách tổng thể cả câu, điều này làm chúng hiệu quả trong các tasks liên quan tới sentence similarity và clustering. Điều này trái ngược với những mô hình token-level như BERT, tập trung vào việc hiểu và nắm bắt ngữ nghĩa cho từng token/ word trong câu của chúng. Chính vì thế, các model như BERT, Roberta, ... sẽ hiệu quả cho những bài toán như NER, ... khi được benchmark với các mô hình dạng sentence-level.

- **Áp dụng Sentence Transformers cho RAG**
    - Có thể nói rằng, Sentence Transformers là lựa chọn số 1 cho mô hình embedding cho RAG.
    - RAG tận dụng Sentence Transformers để có khả năng hiểu và so sánh nội dụng ngữ nghĩa của câu. Việc tích hợp Sentence Transformers đặc biệt tốt trong trường hợp mà các mô hình retrieval cần truy xuất thông tin hiệu quả giữa query và documents trước khi sinh ra câu trả lời.

1. **Improved Document Retrieval:** Sentence Transformers được huấn luyện để tạo ra embeddings mà nắm bắt được nghĩa của câu. Trong RAG, các embeddings này được sử dụng để so khớp một câu query (câu hỏi của user) với documents hoặc các passages trong database. Điều này rất quan trọng vì chất lượng của phản hồi được tạo ra bởi hệ thống phụ thuộc lớn với thông tin được trích xuất.

2. **Efficient Semantic Search:** Các mô hình tìm kiếm thông tin dựa trên keyword có thể gặp vấn đề trong việc hiểu ngữ cảnh, hay sắc thái ngữ nghĩa của query. Sentence Transformers cung cấp embeddings nắm bắt tốt về mặt ngữ nghĩa cho phép tìm kiếm theo sắc thái ngữ nghĩa tốt hơn ngoài việc kết hợp keyword. Điều này có nghĩa là cho phép khả năng truy xuất được thông tin documents liên quan về mặt ngữ nghĩa với câu query ngay cả khi chúng không chứa các keyword chính xác nào.

3. **Contextual Understanding for Better Responses:** Bằng cách sử dụng Transformers, mô hình RAG có thể hiểu rõ cả ngữ cảnh và sắc thái của cả câu query và nội dung của quan trọng của documents. Điều này dẫn tới đưa ra phản hồi chính xác hơn và phù hợp với ngữ cảnh hơn vì việc sinh ra câu trả lời có thông tin được đưa vào phù hợp hơn.

4. **Scalability in Information Retrieval:** Sentence Transformers có thể xử lý hiệu quả với database chứa documents lớn bằng cách tính toán trước embeddings cho toàn bộ documents. Điều này làm cho quá trình retrieval trở nên nhanh hơn và có khả năng mở rộng hơn vì mô hình chỉ cần tính toán tính toán embedding cho câu query trong quá trình chạy và nhanh chóng tìm được các documents có embeddings gần nhất với nó.

5. **Enhancing the Generation Process:** Trong hệ thống RAG, hiệu quả quá trình tìm kiếm được thông tin liên quan tác động trực tiếp tới module sinh (generation) để tạo ra được câu phản hồi. Chính vì thế khả năng có thể truy xuất được thông tin hiệu quả, nắm bắt được ngữ cảnh của retrieval sẽ cho phép mô hình ngôn ngữ tạo ra được các phản hồi không chỉ chính xác về ngữ nghĩa mà còn được cung cấp những thông tin mà nó có thể nằm ngoài những gì mô hình đã được học.

Tóm lại, Sentence Transformers nâng cao khả năng retrieval của mô hình RAG với LLM bằng cách cho phép tìm kiếm và truy xuất thông tin theo ngữ nghĩa hiệu quả hơn. Điều này giúp cho cải thiện hiệu suất trong các tasks mà yêu cầu đòi hỏi phải hiểu và sinh ra câu phản hồi dựa trên khối lượng dữ liệu tri thức văn bản lớn, chẳng hạn như Question Answering, Chatbots, Information Extraction.

Retrieval

- Trong phần này chúng ta sẽ tìm hiểu 3 kiểu retrieval được phân loại dựa trên cách tiếp cận gồm: Standard, Sentence window và Auto-merging. Mỗi cách tiếp cận này đều có điểm mạnh và điểm yếu cụ thể và mức độ phù hợp của chúng phụ thuộc vào yêu cầu của nhiệm vụ RAG, bao gồm tính chất của tập dữ liệu, độ phức tạp của câu query và sự cân bằng 2 khía cạnh về tính cụ thể (Specificity) và hiểu biết ngữ cảnh (contextual) trong việc xử lý và phản hồi các câu truy vấn đó.
    - **Specificity (Độ cụ thể):** Đây là khả năng của một hệ thống trong việc cung cấp câu trả lời chính xác và cụ thể cho một truy vấn. Một hệ thống có độ cụ thể cao sẽ cung cấp thông tin rất chính xác, thường là chi tiết và trực tiếp liên quan đến câu hỏi đặt ra. Ví dụ, khi được hỏi về một sự kiện lịch sử cụ thể, hệ thống sẽ cung cấp thông tin chính xác về sự kiện đó mà không lạc đề.
    - **Contextual Understanding (Hiểu biết ngữ cảnh):** Đây là khả năng của hệ thống trong việc hiểu và tích hợp thông tin ngữ cảnh vào câu trả lời. Một hệ thống có khả năng hiểu biết ngữ cảnh tốt không chỉ trả lời dựa trên thông tin cụ thể được đề cập trong câu hỏi, mà còn xem xét các yếu tố như bối cảnh của câu hỏi, ý định ngầm của người hỏi, và các thông tin liên quan mà không được trực tiếp nêu ra. Điều này giúp cho câu trả lời có độ sâu và liên quan hơn.

Standard/ Naive Approach

- Cách tiếp cận cơ bản thì người ta sẽ sử dụng cùng 1 đoạn văn bản (gọi là chunk) để indexing, embedding cũng như sử dụng để đưa ra output (câu phản hồi).

- Ưu điểm và nhược điểm của phương pháp này khi sử dụng trong RAG:
    - **Ưu điểm:**

1. **Đơn giản và hiệu quả:** Phương pháp này đơn giản và hiệu quả, sử dụng dùng đoạn văn (chunk) cho cả embedding, synthesis (tổng hợp ra câu phản hồi), đơn giản hóa quá trình retrieval.
2. **Đồng nhất trong xử lý dữ liệu:** Nó duy trì tính nhất quán trong dữ liệu được sử dụng trong cả 2 giai đoạn retrieval và synthesis.

- **Nhược điểm:**
    1. **Hạn chế trong hiểu ngữ cảnh:** LLMs yêu cầu ngữ cảnh đầy đủ hơn cho việc tổng hợp ra phản hồi tốt hơn nhưng phương pháp này lại không thể cung cấp đầy đủ được.
    2. **Các phản hồi được sinh ra có thể không tối ưu:** Do ngữ cảnh bị hạn chế, chính vì thế LLM có thể không có đủ thông tin để tạo ra phản hồi chính xác và phù hợp nhất.

Sentence-Window Retrieval / Small-to-Large Chunking

- Cách tiếp cận sentence-window là chia nhỏ documents thành các đơn vị nhỏ, chẳng hạn như câu hoặc nhóm nhỏ các câu (smaller-chunks).
- Nó tách riêng embeddings cho task retrieval (là các chunks nhỏ hơn được lưu trữ trong Vector DB) nhưng nó sẽ được thêm lại vào ngữ cảnh xung quanh của các smaller-chunks được truy xuất. Bạn xem hình ảnh để hiểu rõ hơn bên dưới.

- Trong quá trình retrieval, chúng ta truy xuất các câu phù hợp nhất với câu query thông qua tìm kiếm độ tương đồng và thay thế câu được truy xuất bằng cách bổ sung ngữ cảnh

xung quanh nó (sử dụng sentence-window để lấy ngữ cảnh xung quanh câu được truy xuất ban đầu)

- Ưu điểm và nhược điểm của phương pháp này khi sử dụng trong RAG:

- **Ưu điểm:**
    1. **Nâng cao độ cụ thể (specificity) trong Retrieval:** Mức độ cụ thể hiểu như thế nào bạn có quay lại phần giải thích phía trên của mình. Ở đây, bằng cách chia documents thành các đơn vị nhỏ (smaller-chunk), cho phép truy xuất chính xác hơn các phân đoạn có liên quan trực tiếp tới câu query.
    2. **Giàu ngữ cảnh hơn cho quá trình tổng hợp:** Nó được bổ sung lại ngữ cảnh xung quanh các đoạn được truy xuất cho quá trình synthesis, cung cấp cho LLM thông tin rộng hơn để sinh ra câu phản hồi.
    3. **Cân bằng:** Phương pháp này tạo ra sự cân bằng giữa truy xuất có tính chất chọn lọc, chính xác, nhanh chóng và giữ được nhiều thông tin (giàu ngữ cảnh), có khả năng cải thiện chất lượng quá trình tổng hợp sinh ra câu phản hồi.
- **Nhược điểm:**
    1. **Độ phức tạp cao hơn:** Việc tách biệt cho retrieval, synthesis làm tăng độ phức tạp cho pipeline.
    2. **Vẫn còn tồn tại trường hợp thiếu context:** Vẫn có những tỷ lệ nhỏ nào đó thiếu ngữ cảnh nếu thông tin được bổ sung xung quanh không đủ bao quát.

Auto-merging Retriever / Hierarchical Retriever

- Cách tiếp cận auto-merging retrieval không truy xuất các chunks thiếu tính phân mảnh như cách tiếp cận thông thường (naive approach) ở trên. Sự phân mảnh trong cách tiếp cận thông thường sẽ trở nên tệ khi kích thước chunk càng nhỏ.
- Cách auto-merging retrieval hoạt động chúng ta có thể xem ảnh bên dưới.

- Auto-merging retrieval mục đích kể kết hợp (merge) thông tin từ nhiều nguồn hoặc nhiều đoạn văn bản để tạo ra để tạo ra phản hồi đầy đủ, phù hợp với ngữ cảnh hơn cho một câu query. Phương pháp này đặc biệt hữu ích khi không có tài liệu hay phân đoạn nào của văn bản có thể trả lời đầy đủ ý của câu query mà câu trả lời phải kết hợp thông tin từ nhiều nguồn/ phân đoạn lại với nhau.
- Nó cho phép các chunks nhỏ hơn được merge thành các chunks to hơn. Nó thực hiện thông qua các bước như sau:
    1. Xác định hệ thống phân cấp của các chunks nhỏ (children) được liên kết với chunks lớn hơn (parent-chunk).
    2. Nếu tập các chunk nhỏ liên kiết với cha của nó (parent-chunk) vượt qua một ngưỡng nào đó (chẳng hạn như consine similarity), thì sẽ merge các chunk nhỏ hơn đó vào parent-chunk của nó.
- Phương pháp này cuối cũng sẽ retrieval tới parent-chunk để có context tốt hơn.
- Ưu điểm và nhược điểm của phương pháp này khi sử dụng trong RAG:
    - **Ưu điểm:**
        1. **Phản hồi theo ngữ cảnh toàn diện hơn:** Nhờ cách merge thông tin từ nhiều nguồn sẽ tạo ra các phản hồi đầy đủ, toàn diện hơn và phù hợp hơn với ngữ cảnh.
        2. **Giảm phân mảnh:** Cách tiếp cận này giải quyết vấn đề truy xuất thông tin bị phân mảnh, phổ biến trong cách tiếp cận đơn giản, đặc biệt với kích thước chunk nhỏ hơn.
        3. **Tích hợp nội dung động:** Tức là nó kết hợp các chunks nhỏ hơn thành lớn hơn từ nhiều nguồn, hay đoạn văn theo một cơ chế dynamic làm cho nhiều thông tin hơn, nâng cao tính phong phú trước khi được cung cấp cho LLM.
    - **Nhược điểm:**
        1. **Sự phức tạp trong việc phân cấp và xác định ngưỡng**: Quá trình xác định phân cấp và thiết lập các ngưỡng thích hợp để hợp nhất là phức tạp và quan trọng để hoạt động hiệu quả.
        2. **Rủi ro khái quát hóa quá mức cần thiết:** Có khả năng hợp nhất quá nhiều thông tin hoặc thông tin không liên quan, dẫn đến phản hồi quá rộng hoặc lạc đề.

3. **Mức độ tính toán:** Phương pháp này có thể cần nhiều tính toán hơn do có thêm các bước trong việc hợp nhất và quản lý cấu trúc phân cấp của các đoạn văn bản.

Làm thế nào để tìm ra được kích thước chunk lý tưởng?

- Nhìn chung, khi xây dựng một hệ thống RAG có nhiều tham số, chiến lược từ kích thước chunk tới vector embedding nào, hay phương pháp search,... Trong đó việc xác định được kích thước chunk lý tưởng cho documents có tác động mật thiết tới hệ thống RAG. Kích thước chunk lý tưởng sẽ phụ thuộc vào một số yếu tố:

    1. **Đặc điểm dữ liệu:** Bản chất của dữ liệu của bạn rất quan trọng. Đối với tài liệu văn bản text (documents), chúng ta hãy xem xét độ dài trung bình của các đoạn văn hoặc các phần trong tài liệu. Nếu tài liệu có cấu trúc tốt với các phần tách biệt nhau thì việc phân tách chunks theo 1 cách tự nhiên, thông thường có thể là giải pháp tốt.

    2. **Ràng buộc của mô hình retrieval:** Loại mô hình retrieval mà chúng ta lựa chọn (như BM25, TF-IDF, Neural Network (DPR)) có thể có những hạn chế về độ dài đầu vào (max input length). Chính vì thế phải đảm bảo các chunks tương thích với ràng buộc này là cần thiết để có thể mô hình hóa đầy đủ ngữ nghĩa hơn.

    3. **Bộ nhớ và tài nguyên tính toán:** Khi kích thước chunk lớn thì dẫn tới việc bộ nhớ sử dụng và tài nguyên tính toán sẽ lớn hơn. Cân bằng kích thước chunk và tài nguyên sẵn có cũng là khía cạnh để đảm bảo quá trình xử lý trong hệ thống hiệu quả.

    4. **Yêu cầu từ tác vụ:** Bản chất của các task như Question Answering, document summarization, ... có thể bị ảnh hưởng bởi kích thước chunk. Đối với các task yêu cầu chi tiết, kích thước chunk nhỏ có thể hiệu quả hơn để nắm bắt các chi tiết cụ thể. Trong khi các task rộng, kiểu như những task mà chúng ta cần nhìn một cách tổng quát, khái quát mà không cần tập trung vào từng chi tiết nhỏ để giải quyết thì những task như vậy thì kích thước chunk lớn hơn thường hiệu quả hơn vì nắm bắt được nhiều ngữ cảnh hơn.

    5. **Thử nghiệm:** Một cách mà luôn áp dụng trong mọi thứ khi tiếp cận với AI là thử nghiệm. Vì vậy, cách tốt nhất để xác định được kích thước lý tưởng cho việc chunking là thông qua thử nghiệm và đánh giá.

    6. **Xem xét overlap giữa các chunks:** Đôi khi trong một số bài toán, khi chúng ta overlap giữa các chunks để đảm bảo rằng không có thông tin quan trọng nào bị

bỏ sót ở ranh giới. Việc có nên overlap không chúng ta cần dựa trên đặc điểm của tasks chúng ta muốn làm và đặc tính của dữ liệu.

Tóm lại, việc xác định kích thước lý tưởng cho việc chunking trong hệ thống RAG là quan trọng, cần cân bằng giữa xem xét các điểm điểm của dữ liệu, các hạn chế của mô hình retrieval chúng ta sử dụng, về resources, các yêu cầu cụ thể của tasks và thử nghiệm đánh giá thực tiễn. Quá trình này có thể lặp đi lặp lại, tinh chỉnh để đạt được kết quả tốt nhất.

Ensemble retriever và Reranking

- Chúng ta có thể thử nhiều kích thước chunk cùng 1 lúc, và sử dụng phương pháp ranking nào đấy để lấy kết quả cuối cùng. Cách thức này có 2 mục đích:
    - Kết quả retrieval tốt hơn bằng cách gộp các quả từ nhiều kích thước chunk khác nhau, và ranking lại chúng một cách hiệu quả.
    - Đây cũng là 1 cách để benchmark được các chiến lược retrieval khác nhau dựa trên sự thay đổi kích thước chunk.
- Quá trình này sẽ như sau:
    - Chia nhỏ cùng một tài liệu theo nhiều cách khác nhau, ví dụ như với các kích thước chunk: 128, 256, 512 và 1024.
    - Trong quá trình retrieval, chúng ta tìm các chunk có liên quan từ mỗi bộ retriever, sau đó tập hợp chúng lại với nhau để truy xuất.
    - Cuối cùng, sử dụng bộ ranker để rank lại kết quả
    - Bạn có thể hình dung quá trình như ảnh dưới đây.

- LlamaIndex cũng có 1 thử nghiệm đánh giá các metrics dựa trên việc thay đổi kích thước chunk ở đây. Theo như kết quả này thì chúng ta có thể thấy về độ đo faithfulness có tăng lên một chút với cách ensemble cho thấy kết quả được truy xuất có liên quan hơn một chút. Nhưng ở kết quả pairwise comparision có kết quả ưu tiên như nhau giữa Ensemble với Base-retriever nên vẫn làm cho chúng ta đặt câu hỏi liệu Ensemble có thực sự tốt hơn không?
- Chúng ta cũng lưu ý rằng, việc ensemble có thể áp dụng cho nhiều khía cạnh của hệ thống RAG không chỉ nằm ở việc thay đổi kích thước chunk. Có thể chẳng hạn như về vector embedding với keyworkd hay với hybird searc, ...

Response Generation / Synthesis

- Bước cuối cùng trong hệ thống RAG là quá trình tạo ra phản hồi cho người dùng. Trong bước này, mô hình tổng hợp các thông tin được truy xuất với tri thức có sẵn trong nó (pre-trained knowledge) để tạo ra các phản hồi mạch lạc và phù hợp với ngữ cảnh. Quá trình này bao gồm việc tích hợp những hiểu biết sâu sắc thu thập được từ nhiều nguồn khác nhau, đảm bảo độ chính xác và mức độ liên quan, đồng thời tạo ra phản hồi không chỉ mang tính thông tin mà còn phù hợp với truy vấn ban đầu của người dùng, duy trì giọng điệu tự nhiên và mang tính trò chuyện.
- Lưu ý rằng, trong quá trình tạo prompt (với top-k chunks được retrieval) cho LLM để tạo phản hồi, việc bố trí một cách có chiến lược để những thông tin quan trọng ở đầu hoặc cuối chuỗi đầu vào có thể nâng cao hiệu quả của hệ thống RAG và do đó làm cho hệ thống hoạt động hiệu quả hơn. Điều này được chứng minh trong nghiên cứu Lost in the Middle: How Language Models Use Long Contexts bạn có thể tìm hiểu thêm.

Evaluation

Precision, Recall, F1-score

- Đánh giá thành phần mô hình retrieval trong hệ thống RAG liên quan đến việc đánh giá mức độ hiệu quả của hệ thống truy xuất thông tin (retriever) liên quan để hỗ trợ tạo ra các phản hồi chính xác và phù hợp với ngữ cảnh. Các độ đo (metrics) đánh giá gồm:
    1. **Precision:** Đo tỷ lệ documents được truy xuất có liên quan đến câu truy vấn hoặc prompt đầu vào. Độ chính xác cao cho thấy hệ thống có hiệu quả trong việc tìm kieemms thông tin liên quan trực tiếp đến nhiệm vụ hiện tại.

2. **Recall:** Đánh giá tỷ lệ tất cả các documents liên quan có trong kho tri thức mà hệ thống truy xuất thành công. Recall cao có nghĩa là hệ thống có khả năng truy xuất đầy đủ, bao quát, không bỏ sót thông tin quan trọng.
3. **F1-score:** Một độ đo kết hợp giữa precision và recall, cung cấp một metric duy nhất cân bằng cả hai metrics trên. Nó đặc biệt hữu ích khi chúng ta cần cân nhắc về cả độ chính xác và khả năng bao quát.
4. **Relevance Ranking:** Việc ranking thường được thực hiện bởi mô hình ranker, chúng ta cũng có thể đánh giá mức độ hệ thống ranking các documents được truy xuất về mức độ liên quan của chúng. Lý tưởng nhất là những documents phù hợp nhất sẽ được rank hạng đầu tiên của danh sách documents truy xuất được bởi hệ thống.
5. **Query Coverage:** Đo lường số lượng câu quyery hoặc prompts mà hệ thống có thể truy xuất được thông tin liên quan. Mức độ coverage cao có nghĩa là hệ thống có khả năng xử lý nhiều topics hoặc đa dạng kiểu query.
6. **Diversity of Retrieved Documents:** Đánh giá xem hệ thống có khả năng truy xuất được nhiều loại, đa dạng kiểu documents hay có xu hướng ưu tiên một loại hoặc nguồn thông tin cụ thể.
7. **Latency:** Thời gian thực hiện cho quá trình retrieval. Trong các ứng dụng thực tế, việc có thể truy xuất nhanh hơn thường quan trọng tới trải nghiệm của người dùng.
8. **Robustness:** Đánh giá mức độ hoạt động của hệ thống retrieval với nhiều loại, nhiều kiểu câu/ cách truy vấn khách nhau, bao gồm những kiểu câu truy vấn không rõ ràng ý, phức tạp.
9. **Error Analysis:** Xác định và phân loại các loại lỗi xảy ra trong quá trình retrieval, chẳng hạn như truy xuất ra documents không liên quan, thiếu document quan trọng, v.v.
10. **User-Centric Metrics:** Liên quan đến nghiên cứu hoặc phản hồi của người dùng để đánh giá cách người dùng đánh giá mức độ liên quan và hữu ích của thông tin được truy xuất.
11. **Document Comprehensiveness:** Điều này đo lường mức độ documents được truy xuất bao gồm nhiều khía cạnh hoặc chủ đề khác nhau liên quan đến truy vấn.

- Sử dụng những metrics trên, chúng ta có thể đánh giá thành phần retrieval trong hệ thống RAG hỗ trợ mục tiêu tổng thể trong việc tạo ra các phản hồi chính xác, liên quan và phù hợp với ngữ cảnh trong LLM. Điều quan trọng là phải cân bằng giữa các metrics này vì việc tập trung quá nhiều về 1 khía cạnh (như Precision chẳng hạn) có thể dẫn tới sự đánh đổi ở những khía cạnh khác (như Recall).

Context Relevance, Groundedness, Answer Relevance

- Trong khi các metrics đánh giá truyền thống hay sử dụng các mô hình học máy như Precision, Recall, F1-score, Relevance ranking và các metrics khác cung cấp một nền tảng cơ sở để đánh giá thành phần retrieval của RAG trong LLM, thì những tiến bộ gần đây đã đưa ra nhiều metrics mới như Context Relevance, Groundedness, và Answer Relevance. Những metrics mới này cung cấp góc nhìn toàn diện hơn nhiều về hiệu suất retrieval của hệ thống RAG, có thể sử dụng thư viện như TruLens để triển khai. Bạn có thể tìm hiểu thêm ở đây. TruLens cung cấp những cách thức triển khai cho các metrics trên cho các ứng dụng LLM, cung cấp phân tích chi tiết, cụ thể về hiệu suất của RAG, tập trung vào sự liên kết theo ngữ cảnh và thực tế của nội dung document được truy xuất và được sinh ra để đưa ra phản hồi đáp ứng được câu query của người dùng. Các metrics đánh giá một cách rộng hơn này cho phép hiểu sâu hơn mức độ hoạt động của hệ thống RAG trong các tình huống thực tế trong đó sự phức tạp của ngữ cảnh và độ chính xác thực tế là quan trọng.
- Minh họa các metrics mới trên có thể được sử dụng để đánh giá hệ thống RAG: Groundedness (còn gọi là, Faithfulness), Answer Relevance, and Context Relevance có thể sẽ dễ hình dung hơn bạn có thể xem hình ảnh bên dưới.

1. **Groundedness (a.k.a. Faithfulness)**

- **Category:** Sự liên kết thực tế và độ tương đồng về ngữ nghĩa.

- **Focus:** Metric này đánh giá sự liên kết thực tế và sự tương đồng về ngữ nghĩa giữa phản hồi của mô hình và các tài liệu được truy xuất. Nó đảm bảo rằng phản hồi được tạo ra phù hợp với ngữ cảnh và có căn cứ thực tế dựa trên thông tin được truy xuất.
- **Measurement Methods:** Groundedness có thể được đánh giá bằng cách sử dụng mô hình Nature Language Inference (NLI), Large Language Model (LLM) và sự kết hợp giữa hệ thống tự động và đánh giá của con người. Công thức như sau:

F=‖V‖‖S‖$F=\|S\|\|V\|$

- Trong đó:
    - ‖V‖‖$V\|$ biểu thị số Số lượng các phát biểu có thể được xác minh hoặc chứng minh là chính xác dựa trên ngữ cảnh.
    - ‖S‖‖$S\|$ biểu thị tổng số lượng các phát biểu được tạo ra từ câu trả lời của mô hình.

Ví dụ như nếu một mô hình tạo ra 10 phát biểu và 6 trong số đó có thể được xác nhận là chính xác dựa trên ngữ cảnh, thì ‖V‖=6‖$V\|$=6 và ‖S‖=10‖$S\|$=10.

- **Evaluation Approach:** Sử dụng Chains of Thought (CoT) prompting để mô phỏng quá trình suy luận, chấm điểm sự phù hợp theo thang điểm 0 hoặc 1 thông qua phương pháp kết hợp giữa các hệ thống tự động (để matching ngữ nghĩa và kiểm tra tính xác thực) và đánh giá của con người.

2. **Answer Relevance**

- **Category:** : Chất lượng phản hồi và mức độ liên quan về mặt ngữ nghĩa.
- **Focus:** Metric này đánh giá mức độ liên quan của câu trả lời do mô hình tạo ra với câu query của người dùng, tập trung vào sự liên kết (alignment) giữa câu query của người dùng và phản hồi được sinh ra. Nó được tính bằng cách tạo ra các câu hỏi tiềm năng từ câu trả lời và đo lường mức độ tương tự giữa chúng với câu hỏi gốc.
- **Measurement Methods:** Đánh giá bằng cách sử dụng các mô hình kiểu BERT, khoảng cách embedding, hoặc bằng LLMs thông qua các phép đánh giá độ tương đồng dựa trên embedding (như phép toán dot-product). Công thức như sau:

AR=1n∑i=1nsim(q,qi)$AR=n1\sum i=1n\text{sim}(q,qi)$

- Trong đó: qq là câu hỏi gốc, qiqi là câu hỏi được tạo ra, và simsim biểu thị độ tương đồng cosine giữa các embeddings đó.
- Ví dụ minh hoạ đầu ra của Answer Relevance như ở hình bên dưới:

3. **Context Relevance**

- **Category:** Sự liên kết theo ngữ cảnh (Contextual alignment) và mức độ liên quan (Relevance).

- **Focus:** Đánh giá mức độ phù hợp của ngữ cảnh hoặc nội dung mà hệ thống RAG truy xuất với câu query của người dùng. Nó đánh giá cụ thể xem thông tin được truy xuất có liên quan và phù hợp với câu query nhất định hay không.
- **Measurement Methods:** Có thể được đánh giá bằng các mô hình kiểu BERT nhỏ hơn, khoảng cách embedding hoặc bằng LLM.
- **Evaluation Approach:** Bao gồm quy trình gồm hai bước:
    - Đầu tiên, xác định các câu có liên quan bằng cách sử dụng semantic similarity để tạo ra relevance score cho mỗi câu và sau đó, đánh giá mức độ liên quan với ngữ cảnh tổng thể. Công thức như sau:

CR=number of extracted sentencestotal number of sentences in c(q)CR=total number of sentences in $c(q)$number of extracted sentences

- Ví dụ minh hoạ đầu ra của Context Relevance như hình ở bên dưới:

Nhìn chung, các metrics này đưa ra một cách tiếp cận kiểu đa góc nhìn mới, đa chiều hơn để đánh giá hệ thống RAG, nhấn mạnh không chỉ việc truy xuất thông tin mà còn cả về khía cạnh mức độ liên quan theo ngữ cảnh, độ chính xác thực tế và sự liên kết ngữ nghĩa với câu query của người dùng.

Nguồn tham khảo

- Chunking Strategies for LLM Applications, https://www.pinecone.io/learn/chunking-strategies/.
- Building and Evaluating Advanced RAG, https://learn.deeplearning.ai/building-evaluating-advanced-rag/lesson/1/introduction.
- Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, https://arxiv.org/abs/2005.11401.
- Retrieval-Augmented Generation for Large Language Models: A Survey, https://arxiv.org/abs/2312.10997.
- Retrieval-Augmented Generation (RAG): From Theory to LangChain Implementation, https://towardsdatascience.com/retrieval-augmented-generation-rag-from-theory-to-langchain-implementation-4e9bd5f6a4f2.